

arc42-Architekturdokumentation — LoL Coach

Version: 1.0 **Datum:** 2026-06-06 **Status:** In Arbeit **Autoren:** NexusHero

Diese Dokumentation folgt der [arc42-Vorlage](#) in der Markdown-Variante gemäß iSAQB-Empfehlung.

Inhaltsverzeichnis

- 1. [Einführung und Ziele](#)
- 2. [Randbedingungen](#)
- 3. [Kontextabgrenzung](#)
- 4. [Lösungsstrategie](#)
- 5. [Bausteinsicht](#)
- 6. [Laufzeitsicht](#)
- 7. [Verteilungssicht](#)
- 8. [Querschnittliche Konzepte](#)
- 9. [Architekturentscheidungen](#)
- 10. [Qualitätsanforderungen](#)
- 11. [Risiken und technische Schulden](#)
- 12. [Glossar](#)

1. Einführung und Ziele

1.1 Aufgabenstellung

LoL Coach ist ein KI-gestützter Item-Berater für das Spiel *League of Legends*. Das System liest während eines laufenden Spiels Echtzeit-Daten aus der lokalen Riot-Games-Live-Client-API aus, erkennt spielrelevante Ereignisse und gibt dem Spieler kontextbezogene Item-Empfehlungen — entweder regelbasiert oder durch eine KI-generierte Erklärung angereichert.

Das System löst folgendes Problem: Während eines Spiels müssen Spieler in kurzer Zeit entscheiden, welche Items sie als Nächstes kaufen. Diese Entscheidung hängt von der gegnerischen Teamzusammensetzung ab (AP/AD-Verhältnis, CC-Dichte, Heilchampions) und überfordert besonders unerfahrene Spieler.

1.2 Qualitätsziele

Priorität	Qualitätsmerkmal	Ziel
1	Verfügbarkeit	Empfehlungen sind auch ohne Internetzugang verfügbar (Heuristik-Modus)
2	Reaktionszeit	Heuristische Empfehlung in < 5 ms; LLM-Erklärung in < 3 s
3	Sicherheit	API-Keys werden im OS-Keychain gespeichert, nicht im Klartext
4	Änderbarkeit	Champion- und Item-Daten können ohne Code-Deployment aktualisiert werden
5	Testbarkeit	Alle Kernkomponenten sind durch Unit-Tests abgedeckt; Abhängigkeiten sind injizierbar

1.3 Stakeholder

Rolle	Erwartung
Spieler	Schnelle, verständliche Item-Empfehlungen während des Spiels
Entwickler	Klare Komponentenstruktur, einfache Erweiterbarkeit um neue LLM-Anbieter
Betreiber (Selbsthosting)	Einfache Installation als eigenständige .exe/.app ohne externe Abhängigkeiten

2. Randbedingungen

2.1 Technische Randbedingungen

Randbedingung	Hintergrund
Riot Live Client API läuft auf https://127.0.0.1:2999 mit selbstsigniertem Zertifikat	Die API ist ein nicht-dokumentierter, lokaler HTTP-Server des LoL-Clients. TLS-Verifikation muss deaktiviert werden.
Die Bridge muss als eigenständige ausführbare Datei distribuierbar sein	Endnutzer haben keine Node.js-Laufzeit. Der Bundle wird via @yao-pkg/pkg in ein natives Binary kompiliert.
Die Flutter-App muss auf Windows, macOS und Linux laufen	Desktop-only; mobile Unterstützung ist nicht geplant.
LLM-API-Aufrufe entstehen Kosten	Die Architektur muss einen Cooldown-Mechanismus erzwingen und Tokens durch State-Minifcation minimieren.
Keine persistente Backend-Infrastruktur	Das gesamte System läuft lokal auf dem Rechner des Spielers. Kein Cloud-Backend, keine Datenbank.

2.2 Organisatorische Randbedingungen

Randbedingung	Hintergrund
Open-Source-Projekt (MIT-Lizenz)	Alle verwendeten Bibliotheken müssen mit MIT kompatibel sein. GPL/AGPL sind ausgeschlossen.
CI/CD über GitHub Actions	Security-Scan (npm audit), License-Check und Build-Pipeline laufen bei jedem Push.
Kein dediziertes Ops-Team	Die Architektur muss wartungsarm sein; Monitoring ist nicht eingeplant.

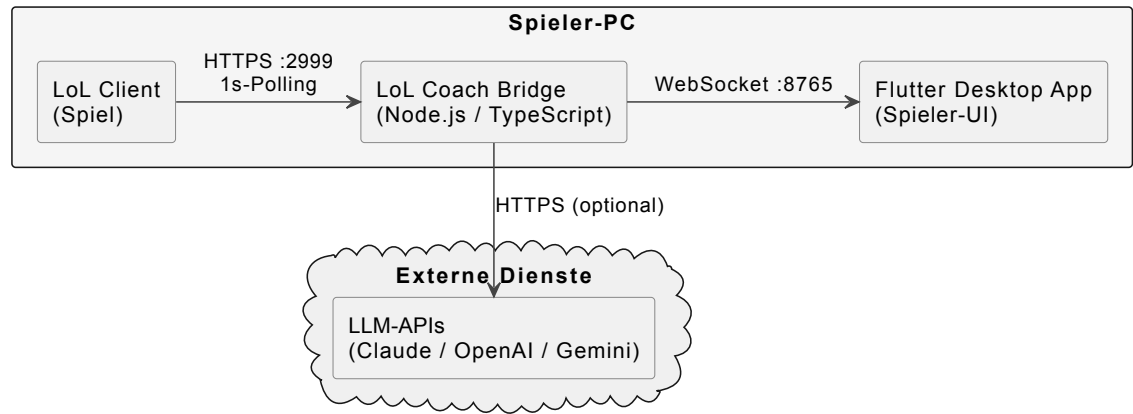
2.3 Konventionen

Konvention	Anwendung
Conventional Commits	Alle Git-Commits folgen dem Format type(scope): message

TypeScript strict mode	Aktiviert in tsconfig.json; kein implizites any
very_good_analysis	Flutter Lint-Preset für konsistente Dart-Code-Qualität
arc42 (Markdown)	Diese Dokumentation

3. Kontextabgrenzung

3.1 Fachlicher Kontext



Nachbarsystem	Beziehung	Richtung
Riot Live Client API	Liefert Echtzeit-Spielzustand (alle Spieler, Items, Gold, Spielzeit)	Bridge → API (Pull, 1 s Intervall)
Claude API (Anthropic)	Generiert natürlichsprachige Item-Erklärungen	Bridge → Claude (Push, optional, mit Cooldown)
OpenAI API	Alternativer LLM-Anbieter	Bridge → OpenAI (Push, optional)
Google Gemini API	Alternativer LLM-Anbieter	Bridge → Gemini (Push, optional)
Flutter Desktop App	Empfängt Events und Empfehlungen; sendet Konfigurationsänderungen	Bridge ↔ App (WebSocket, bidirektional)

3.2 Technischer Kontext

Kanal	Protokoll	Format	Port
Bridge → Riot API	HTTPS (GET)	JSON	2999
Bridge → LLM-APIs	HTTPS (POST)	JSON (SDK)	443
Bridge ↔ Flutter App	WebSocket	JSON (AsyncAPI 2.6)	8765 (konfigurierbar)

Das WebSocket-Protokoll ist in `bridge/asyncapi.yml` vollständig spezifiziert.

4. Lösungsstrategie

Zentrale Architekturentscheidungen auf einen Blick

Problem	Entscheidung	Begründung
Wie werden Spielereignisse erkannt?	Zustandsvergleich zwischen zwei aufeinanderfolgenden Polls	Riot stellt keine Event-API bereit; Polling ist die einzige Option
Wie werden Empfehlungen generiert?	Heuristik-first, LLM-optional	Heuristik ist sofort verfügbar (< 5 ms), kein Internet nötig; LLM enriched nur bei Bedarf
Wie werden mehrere LLM-Anbieter unterstützt?	Adapter-Pattern mit gemeinsamen LlmProvider-Interface	Neue Anbieter erfordern nur eine neue Datei
Wie wird die App auf den Spieler-PC gebracht?	Bridge als natives Binary (pkg), Flutter als .exe/.app/.deb	Keine Installation von Node.js oder Dart erforderlich
Wie wird der Prozesslebenszyklus verwaltet?	Bridge erhält beim Start die Parent-PID der Flutter-App	Bridge beendet sich automatisch, wenn die App geschlossen wird
Wie werden API-Keys sicher gespeichert?	flutter_secure_storage (OS-Keychain)	Verhindert Klartext-Speicherung im Dateisystem

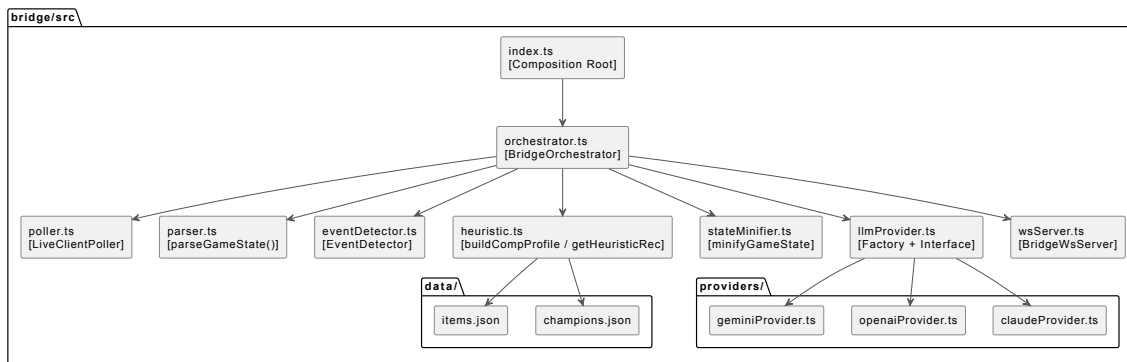
5. Bausteinsicht

5.1 Ebene 1 — Gesamtsystem



Baustein	Verantwortung
Bridge	Datenbeschaffung, Ereigniserkennung, Empfehlungsgenerierung, WebSocket-Server
Flutter App	Benutzeroberfläche, Verbindungsverwaltung, Darstellung von Spielzustand und Empfehlungen

5.2 Ebene 2 — Bridge (Whitebox)



Schnittstellenübersicht Bridge-intern:

Schnittstelle	Von	Nach	Typ
DataFetcher	LiveClientPoller	extern (Riot API)	Async-Funktion
AllGameData	LiveClientPoller	BridgeOrchestrator	TypeScript-Interface (Zod-validiert)
ParsedGameState	parseGameState()	EventDetector, BridgeOrchestrator	TypeScript-Interface
GameEvent[]	EventDetector	BridgeOrchestrator	Discriminated Union
ItemRecommendation	heuristic.ts / LlmProvider	BridgeWsServer	TypeScript-Interface
WsMessage	BridgeWsServer	Flutter App	JSON over WebSocket

5.3 Ebene 2 — Flutter App (Whitebox)

```

flutter_app/lib/
├── main.dart          ← Entry Point, Provider-Setup, Bridge-Autostart
├── services/
│   └── ws_service.dart ← [WsService extends ChangeNotifier]
│                       WebSocket-Client, State-Machine, Auto-Reconnect
├── screens/
│   ├── main_screen.dart ← [MainScreen] Bottom-Navigation, Snackbar-Fehler
│   └── home_screen.dart  ← [HomeScreen] Coach-Tab, Status-Routing
├── widgets/
│   ├── connection_form.dart ← Verbindungseinstellungen, Provider-Auswahl
│   ├── game_view.dart      ← Hauptansicht während Spiel
│   ├── game_top_bar.dart   ← Spielmodus, Zeit, Gold
│   ├── local_player_card.dart ← Eigener Champion, Stats, Items
│   ├── recommendation_panel.dart ← Item-Vorschläge + Erklärung
│   └── scoreboard.dart     ← Teamansicht (Allies / Enemies)

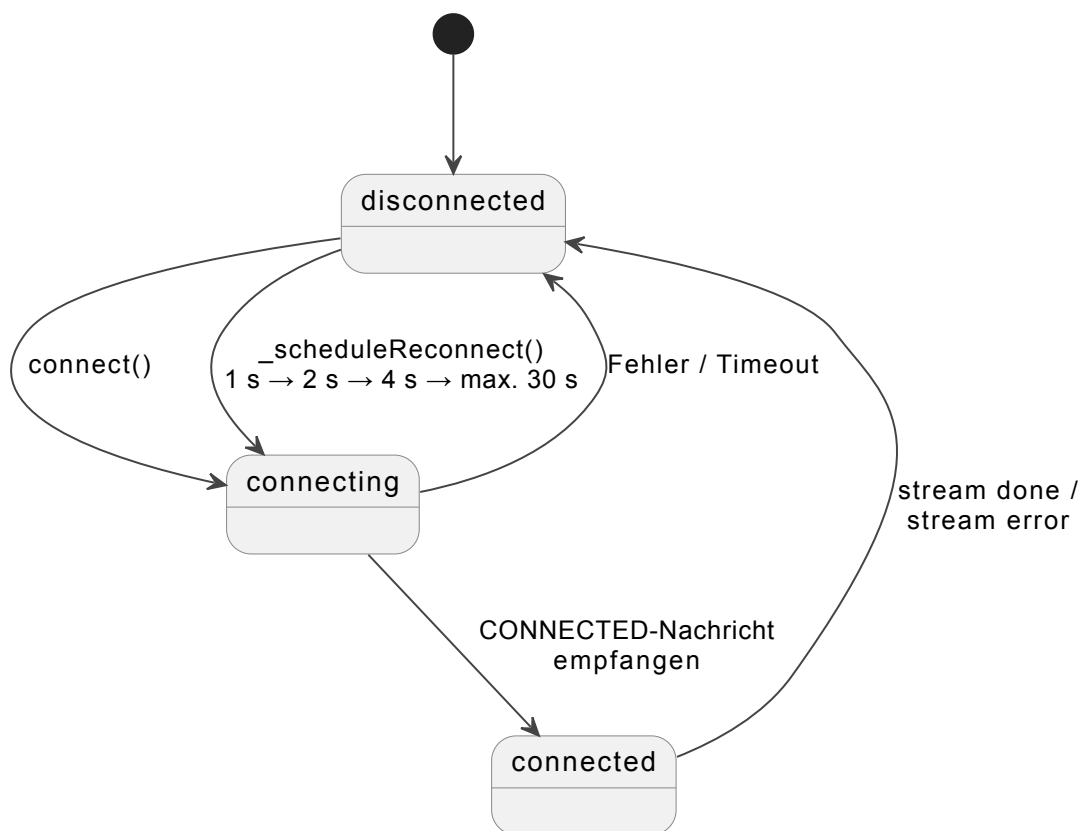
```

```

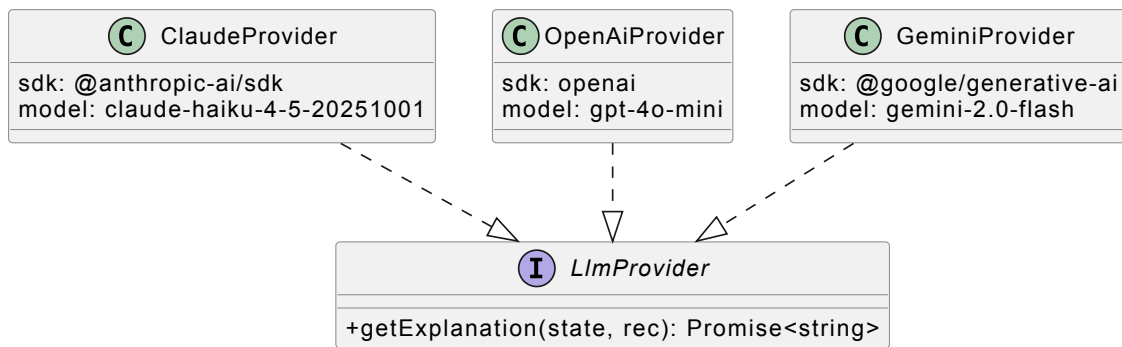
├── shared_widgets.dart      ← Wiederverwendbare UI-Komponenten
├── models/
│   ├── game_state.dart
│   ├── player.dart
│   ├── active_player.dart
│   ├── item.dart
│   ├── recommendation.dart
│   └── ws_message.dart
├── theme/
│   └── app_colors.dart      ← Zentrales Farbschema (LoL-Dark-Theme)

```

State-Machine WsService :



5.4 Ebene 3 — LLM Provider (Whitebox)

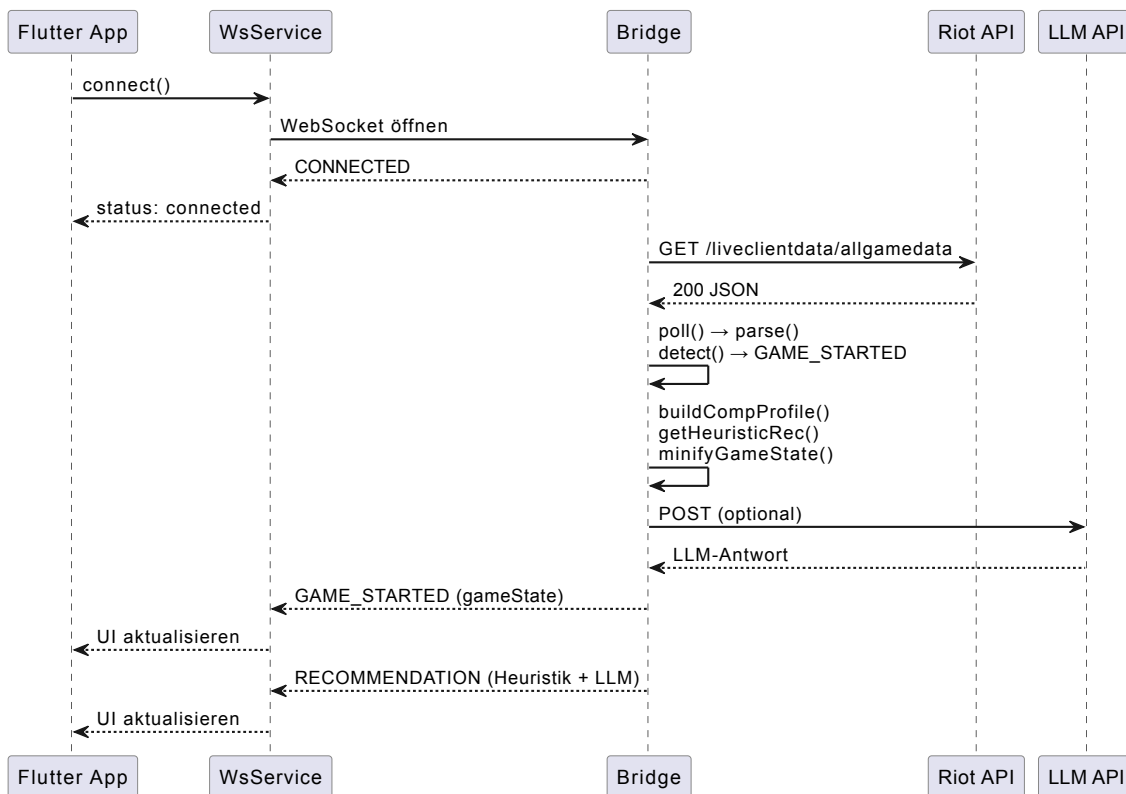


Alle drei Provider:

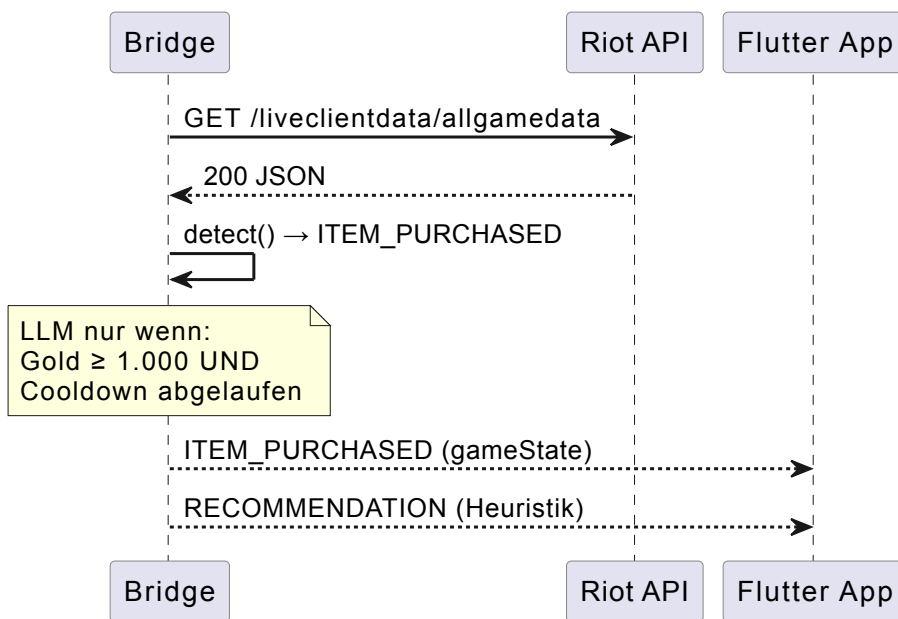
- erhalten denselben `SYSTEM_PROMPT` aus `llmProvider.ts`
- sind auf 150 Output-Tokens limitiert
- fallen bei Fehler auf die heuristische `reasoning` -Zeichenkette zurück

6. Laufzeitsicht

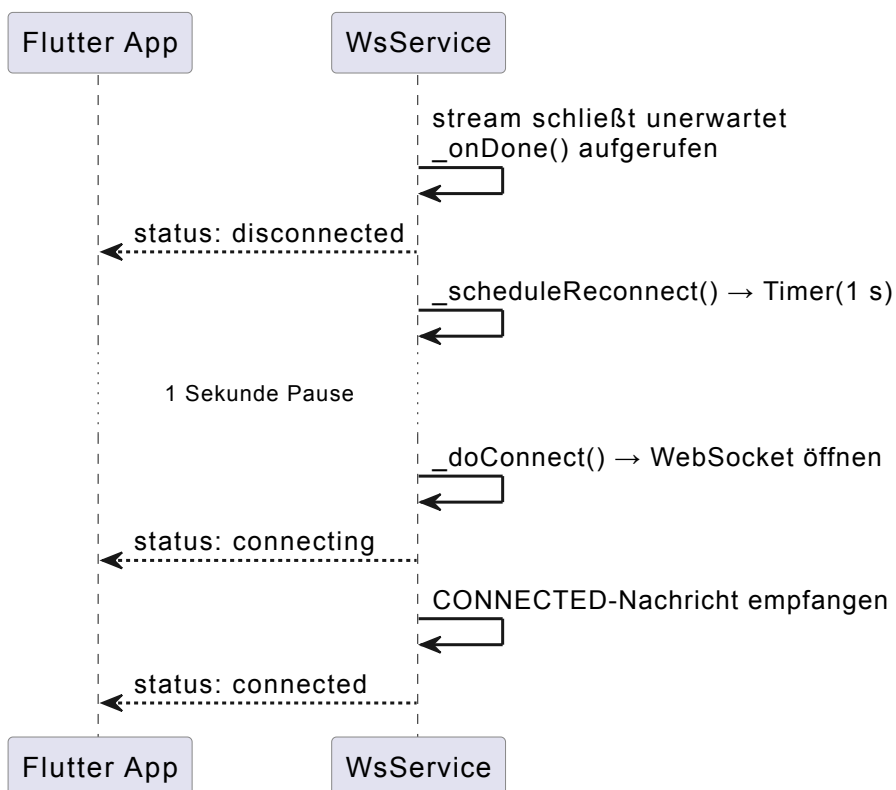
6.1 Szenario: Spielstart mit LLM-Empfehlung



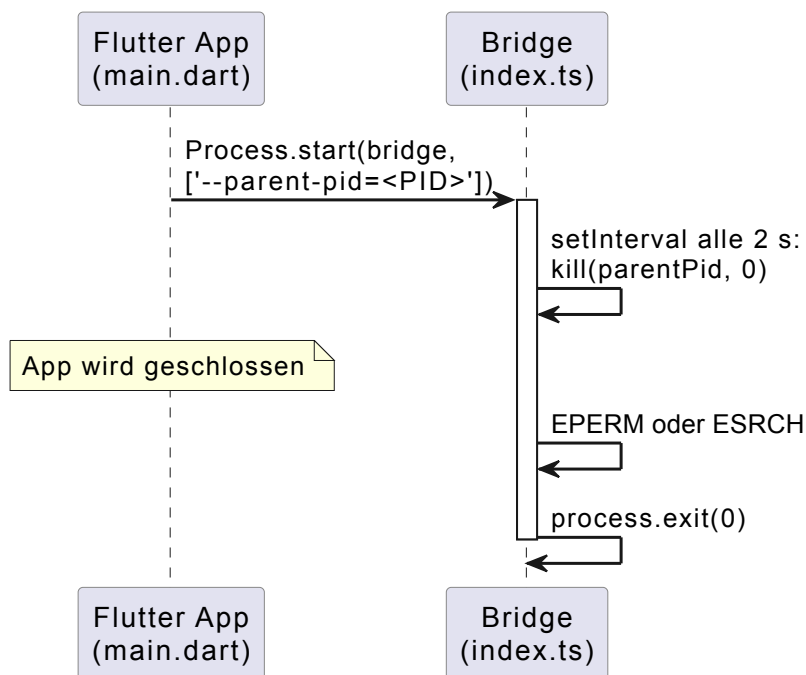
6.2 Szenario: Feind kauft Item (Heuristik-only)



6.3 Szenario: Verbindungsabbruch mit Auto-Reconnect

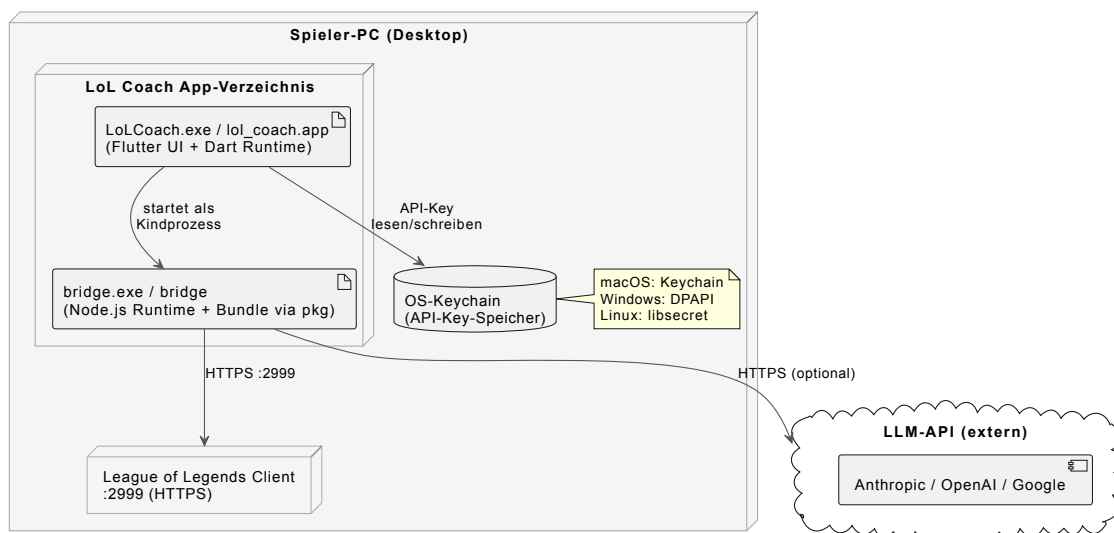


6.4 Szenario: Bridge-Autostart und Prozessüberwachung



7. Verteilungssicht

7.1 Infrastrukturübersicht

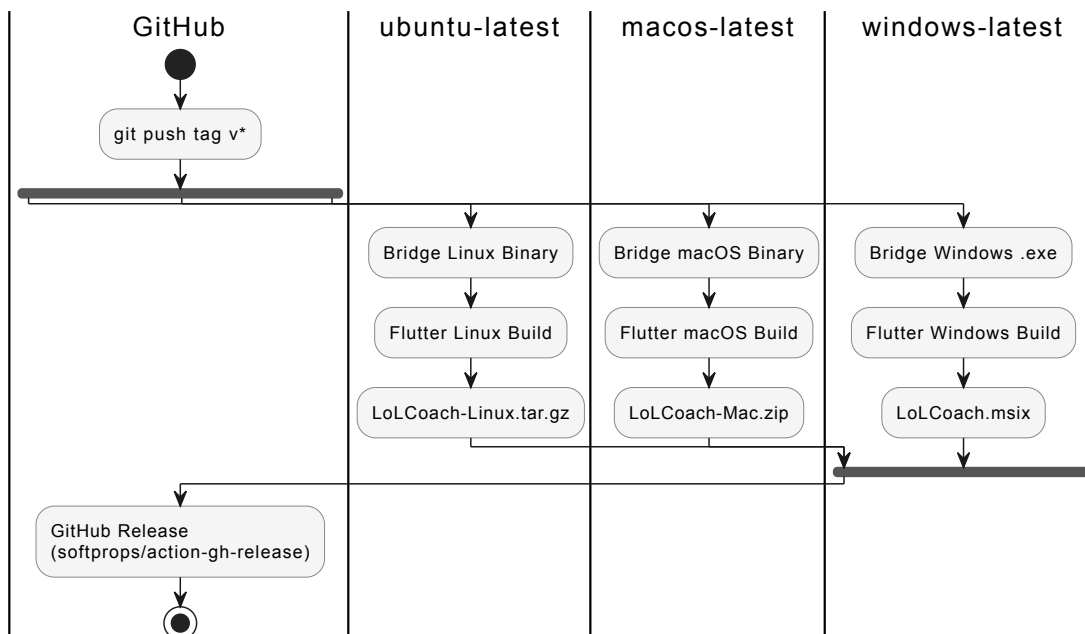


7.2 Deployment-Artefakte

Artefakt	Plattform	Erzeugung	Inhalt
LoLCoach.msix	Windows	deployment/build_installer.ps1	Flutter-App + bridge.exe (via MSIX-Bundle)

LoLCoach-Mac.zip	macOS	deployment/build_unix.sh	lol_coach.app + bridge Binary (im .app-Bundle)
LoLCoach-Linux.tar.gz	Linux	deployment/build_unix.sh	Flutter-Bundle + bridge Binary

7.3 Build-Pipeline (CI/CD)



Zusätzliche Workflows:

Workflow	Trigger	Prüfung
ci.yml	Push / PR	TypeScript-Build, Jest-Tests
security.yml	Push / PR / wöchentlich	npm audit --audit-level=high, License-Check
webpack.yml	Push / PR	webpack-Bundle
docs.yml	Push / PR	TypeDoc-Generierung

8. Querschnittliche Konzepte

8.1 Typsicherheit und Validierung

Das System setzt durchgehend auf statische Typen (TypeScript strict, Dart-Typsystem) und runtime-Validierung an der Systemgrenze.

Strategie: Zod-Schemas validieren alle Daten, die von der Riot-API empfangen werden. Innerhalb der Bridge werden nur typisierte Objekte weitergereicht — kein `any`, kein ungeprüftes JSON.

```
// Eingehende Daten → immer durch Zod-Schema
const parsed = AllGameDataSchema.safeParse(raw);
if (!parsed.success) {
  console.warn("[Poller] Unexpected data format:", parsed.error.issues[0]);
  return; // Verarbeitung abgebrochen, kein Crash
}
// Ab hier: parsed.data ist vollständig typisiert
```

8.2 Fehlerbehandlung

Ebene	Strategie
Riot-API-Polling	Transiente Fehler werden bis zu MAX_POLL_FAILURES = 3 toleriert; erst dann wird GAME_INACTIVE gebroadcastet
LLM-API-Aufruf	Bei Fehler wird auf die heuristische reasoning-Zeichenkette zurückgefallen — kein User-sichtbarer Fehler
WebSocket-Verbindung (Client)	Automatischer Reconnect mit exponentiellem Backoff (1 s → 2 s → 4 s → max. 30 s)
Zod-Validierungsfehler	Verarbeitung der aktuellen Nachricht wird abgebrochen; Polling läuft weiter
JSON-Parse-Fehler (WS)	console.warn mit gekürzter Rohdaten-Ausgabe; Verbindung bleibt offen

8.3 Sicherheit

Maßnahme	Implementierung
API-Key-Speicherung	flutter_secure_storage (macOS Keychain / Windows DPAPI / Linux libsecret)
TLS-Bypass (Riot-API)	rejectUnauthorized: false nur für 127.0.0.1:2999 — lokal, kein Angriffsrisiko
API-Key-Übertragung	Nur über lokale WebSocket-Verbindung (LAN); keine externe Übertragung
Dependency-Sicherheit	npm audit in CI; pkg → @yao-pkg/pkg (CVE GHSA-22r3-9w55-cj54 behoben)
License-Compliance	license-checker-rseidelsohn in CI; GPL/AGPL werden abgelehnt

8.4 Testkonzept

Ebene	Technologie	Abdeckung
Unit-Tests Bridge	Jest + ts-jest	EventDetector, Parser, Heuristik, alle LLM-Provider, Poller, WsServer, Orchestrator, StateMinifier
Integrationstests Bridge	Jest	5 realistische Spielverläufe (Losing/Winning/Scaling)
Unit-Tests Flutter	flutter_test	WsService State-Machine, Model-Deserialisierung

Testinfrastruktur	fixtures.ts, FakeWebSocketChannel	Vorgefertigte Testdaten; kein echter Netzwerkzugriff
-------------------	--------------------------------------	---

Testprinzipien:

- Dependency Injection in allen Kernklassen (Testbarkeit durch Konstruktor-Parameter)
- LLM-Provider werden durch Jest-Mocks ersetzt
- `clock: () => number` in `BridgeOrchestrator` ermöglicht Zeitsteuerung in Tests

8.5 Token-Optimierung (LLM)

Vor jedem LLM-Aufruf wird der Spielzustand durch `minifyGameState()` komprimiert:

```
Eingabe (Riot-API JSON): ~4.000 Zeichen
Ausgabe (minifiziert): ~200 Zeichen

Beispielausgabe:
Time: 12:30
Me: Ahri (Lvl 8, Gold: 1240, KDA: 3/1/2)
My Items: Luden's Tempest, Sorcerer's Shoes
Allies: Jinx (Lvl 7, KDA: 5/0/1), ...
Enemies: Malphite (Lvl 8, KDA: 1/2/0), ...
```

Negative Spielzeiten (Ladebildschirm) werden auf `0:00` geclippt.

8.6 LLM-Cooldown

Um unkontrollierte API-Kosten zu vermeiden, wird ein LLM-Aufruf nur ausgelöst, wenn:

1. Ein LLM-Provider konfiguriert ist
2. Mindestens ein Flutter-Client verbunden ist
3. Das Ereignis `GAME_STARTED` ist — oder — `PLAYER_DIED` **und** `Gold ≥ HIGH_GOLD_THRESHOLD` (1.000) **und** letzte LLM-Anfrage vor mehr als `DEFAULT_LLM_COOLDOWN_MS` (7 Minuten)

8.7 Konfigurierbarkeit

Parameter	Ort	Default
WebSocket-Port	WS_PORT (Umgebungsvariable)	8765
Summoner-Name	SUMMONER_NAME (Umgebungsvariable) oder WS-Nachricht SET_SUMMONER	(leer)
LLM-Anbieter	WS-Nachricht SET_LLM_PROVIDER oder ANTHROPIC_API_KEY (Umgebungsvariable)	Keiner
Champion-Klassifikationen	bridge/src/data/champions.json	Eingebaut
Item-Namen	bridge/src/data/items.json	Eingebaut
Heuristik-Schwellwerte	champions.json → thresholds	Eingebaut

9. Architekturentscheidungen

ADR-001: Heuristik-first mit optionalem LLM

Kontext: Item-Empfehlungen müssen auch ohne Internetzugang und ohne konfiguriertem API-Key funktionieren.

Entscheidung: Die regelbasierte Heuristik-Engine ist immer aktiv und liefert sofort eine Empfehlung. Der LLM-Anbieter ist optional und reichert die Empfehlung bei Bedarf mit einer natürlichsprachigen Erklärung an.

Konsequenzen:

- (+) Offline-Betrieb vollständig funktionsfähig
 - (+) Keine Latenz-Abhängigkeit vom LLM für die Kernfunktion
 - (+) Fallback bei LLM-Fehler ist nahtlos
 - (-) Heuristik-Erklärungen sind weniger kontextbezogen als LLM-Erklärungen
-

ADR-002: Adapter-Pattern für LLM-Anbieter

Kontext: Drei verschiedene LLM-APIs (Anthropic, OpenAI, Google) sollen austauschbar nutzbar sein.

Entscheidung: Gemeinsames `LlmProvider` -Interface mit `getExplanation(state, rec): Promise<string>`. Factory-Funktion `createLlmProvider(type, apiKey)` erzeugt die konkrete Implementierung.

Konsequenzen:

- (+) Neue Anbieter erfordern nur eine neue Datei in `providers/`
 - (+) Orchestrator ist vollständig entkoppelt vom konkreten Anbieter
 - (+) Einfach mockbar in Tests
 - (-) Static-Import aller Provider notwendig (für webpack/pkg-Bundling)
-

ADR-003: WebSocket statt REST-Polling durch Flutter

Kontext: Die Flutter-App muss kontinuierlich Spielzustand-Updates empfangen.

Entscheidung: Die Bridge betreibt einen WebSocket-Server. Flutter ist Client. Events werden gepusht, nicht gepollt.

Konsequenzen:

- (+) Echtzeit-Updates ohne Polling-Overhead auf App-Seite
 - (+) Bidirektionale Kommunikation (`SET_SUMMONER` , `SET_LLM_PROVIDER`)
 - (+) Bridge-seitige Filterung unnötiger Updates möglich
 - (-) Verbindungsmanagement (Reconnect, Zustandswiederherstellung) auf Client-Seite nötig
-

ADR-004: Prozessmodell — Bridge als eigenständiger Kindprozess

Kontext: Die Bridge ist ein Node.js-Prozess; die Flutter-App ist eine Dart-Laufzeit. Beide müssen zusammen starten und enden.

Entscheidung: Die Flutter-App startet die Bridge via `Process.start()` mit `--parent-pid=<PID>`. Die Bridge überwacht die Parent-PID alle 2 Sekunden und beendet sich selbst, wenn der Parent nicht mehr

existiert.

Konsequenzen:

- (+) Kein manueller Bridge-Start für Endnutzer
- (+) Automatisches Cleanup beim App-Schließen
- (-) Auf Windows kann `process.kill(pid, 0)` bei unterschiedlichen Integrity-Levels EPERM werfen (dokumentierter Workaround vorhanden)

ADR-005: Deployment als native Binaries via pkg

Kontext: Endnutzer haben keine Node.js-Laufzeit installiert.

Entscheidung: Die Bridge wird mit `@yao-pkg/pkg` in ein plattformspezifisches Binary kompiliert (`.exe` , macOS-Binary, Linux-Binary). Das Binary enthält die gesamte Node.js-Laufzeit und das webpack-Bundle.

Konsequenzen:

- (+) Zero-Dependency-Installation für Endnutzer
- (+) `pkg` wurde durch `@yao-pkg/pkg` ersetzt (aktiv gepflegt, keine CVE)
- (-) Bundle-Größe ~50 MB durch eingebettete Node.js-Runtime
- (-) Webpack muss mit `minimize: false` konfiguriert sein (pkg-Kompatibilität)

ADR-006: Zod für Runtime-Validierung an der API-Grenze

Kontext: Die Riot Live Client API ist nicht offiziell dokumentiert und kann sich ändern. Fehlende oder falsche Felder würden zu Runtime-Crashes führen.

Entscheidung: Alle eingehenden API-Daten werden durch Zod-Schemas validiert. Fehlende optionale Felder werden durch `.catch(defaultValue)` abgefangen.

Konsequenzen:

- (+) Keine unerwarteten `undefined`-Fehler tief im Code
- (+) Typsicherheit vom Netzwerk bis zur Verarbeitungslogik
- (+) Schema dokumentiert die erwartete API-Struktur
- (-) Leichter Overhead durch Schema-Validierung (vernachlässigbar bei 1 s Polling-Intervall)

ADR-007: Champion- und Item-Daten in JSON-Konfigurationsdateien

Kontext: Champion-Klassifikationen und Item-Namen waren als TypeScript-Konstanten hardcodiert. Meta-Änderungen erforderten ein Code-Deployment.

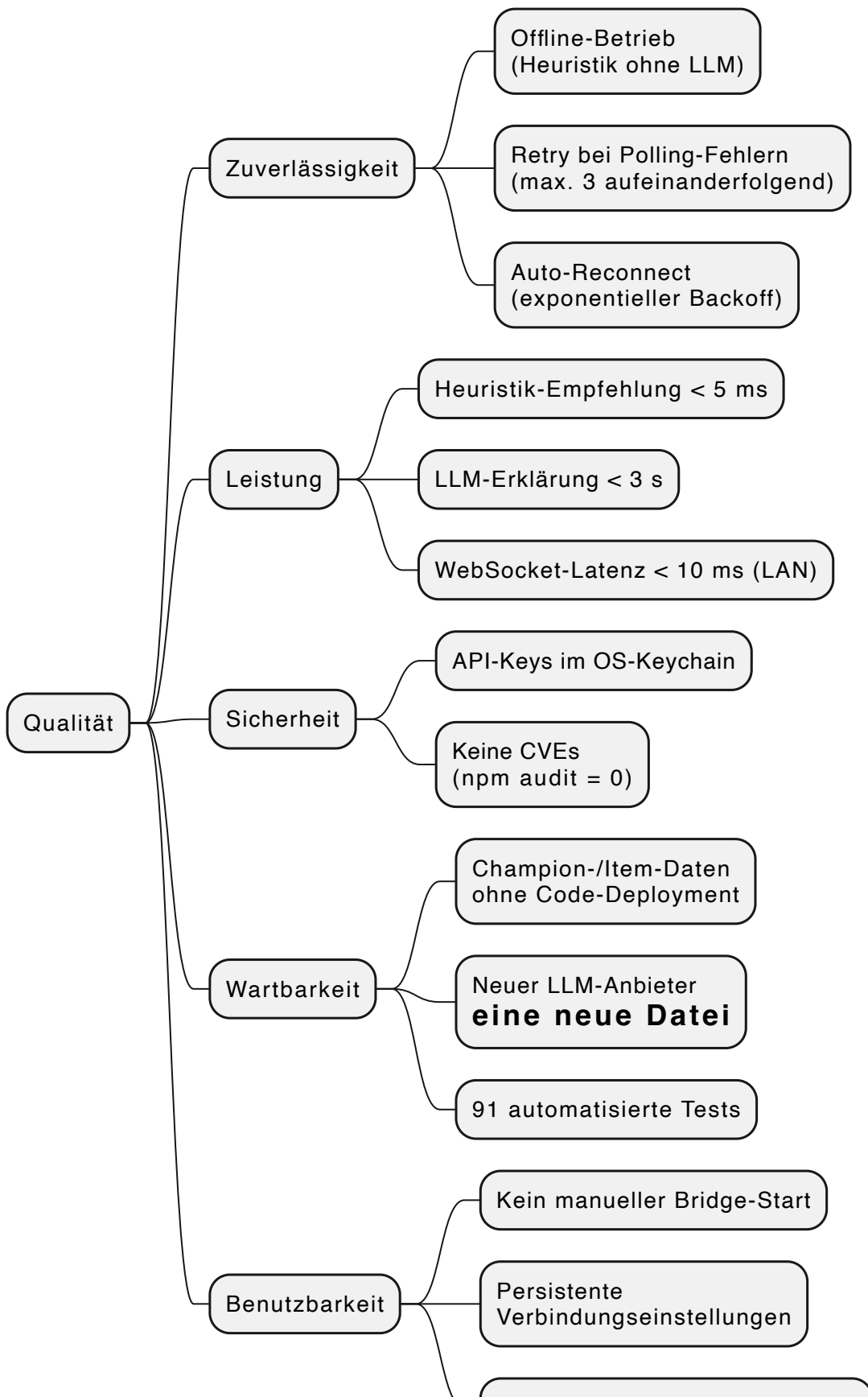
Entscheidung: Auslagern in `bridge/src/data/champions.json` und `bridge/src/data/items.json`. TypeScript importiert die JSON-Dateien statisch (`resolveJsonModule: true`); webpack bundelt sie ins Binary.

Konsequenzen:

- (+) Meta-Updates ohne Code-Änderung möglich (JSON-Datei editieren, neu bundeln)
 - (+) Schwellwerte sind benannt und selbstdokumentierend
 - (-) JSON wird statisch ins Binary gebacken; Laufzeit-Austausch ohne Neustart nicht möglich
-

10. Qualitätsanforderungen

10.1 Qualitätsbaum



10.2 Qualitätsszenarien

ID	Qualitätsmerkmal	Stimulus	Reaktion	Messbar
Q1	Verfügbarkeit	Internetzugang fällt aus	Heuristik liefert weiterhin Empfehlungen	Heuristik-Empfehlung erscheint in < 5 ms
Q2	Leistung	GAME_STARTED-Ereignis tritt auf	Heuristik-Empfehlung wird gebroadcastet	< 5 ms nach Ereigniserkennung
Q3	Fehlertoleranz	Riot-API antwortet 2x nicht	Kein GAME_INACTIVE gebroadcastet	Erst nach 3 aufeinanderfolgenden Fehlern
Q4	Fehlertoleranz	WebSocket-Verbindung bricht ab	App reconnectet automatisch	Erste Reconnect-Versuch nach 1 s
Q5	Sicherheit	Angreifer liest Dateisystem des Nutzers	API-Key ist nicht als Klartext vorhanden	flutter_secure_storage verwendet OS-Keychain
Q6	Wartbarkeit	Riot veröffentlicht neuen Patch (neuer Champion)	Champion-Klassifikation wird aktualisiert	JSON-Datei editieren; kein TypeScript-Deployment
Q7	Testbarkeit	Entwickler fügt neuen LLM-Provider hinzu	Neue Datei in providers/; keine Änderung in Orchestrator	Neuer Provider implementiert LlmProvider-Interface

11. Risiken und technische Schulden

11.1 Risiken

ID	Risiko	Wahrscheinlichkeit	Auswirkung	Maßnahme
R1	Riot ändert die Live Client API ohne Ankündigung (undokumentiert)	Mittel	Hoch — Poller empfängt keine validen Daten mehr	Zod-Schemas schnell anpassen; CI-Alert bei Validierungsfehlern
R2	LLM-API-Kosten steigen	Niedrig	Mittel — Nutzung wird unwirtschaftlich	Cooldown-Mechanismus vorhanden; Anbieter wechselbar

R3	@yao-pkg/pkg wird abandoned	Niedrig	Mittel — kein Binary-Build mehr möglich	Fork oder Wechsel zu nexx als Alternative
R4	Windows-EPERM-Bug im Watchdog	Niedrig	Niedrig — Bridge beendet sich fälschlich bei elevated privileges	Dokumentiert in index.ts; Workaround: Bridge manuell starten
R5	Riot blockiert Live Client API für Drittanwendungen	Niedrig	Sehr hoch — gesamte Kernfunktion bricht weg	Kein technischer Workaround möglich; API-ToS beachten

11.2 Technische Schulden

ID	Schuld	Priorität	Aufwand	Beschreibung
S1	Statische Champion-Klassifikationen	Hoch	Mittel	AP/CC/Healer-Listen decken nicht alle Champions ab und berücksichtigen keine Build-Varianzen (z. B. AP-Corki). Mittelfristig: Daten aus einer Community-API (Data Dragon) laden.
S2	Keine Flutter-Widget-Tests	Hoch	Mittel	GameView, RecommendationPanel, Scoreboard sind ungetestet. Bei Refactorings fehlt das Safety Net.
S3	Kein E2E-Test (Bridge + App)	Mittel	Hoch	Die Integration zwischen Bridge und Flutter ist nur manuell getestet. Ein automatisierter E2E-Test würde Regressions frühzeitig erkennen.
S4	Keine Build-Path-Strategie in build_unix.sh	Niedrig	Niedrig	cd-basierte relative Pfade sind fragil. Wechsel zu absoluten Pfaden via \$(dirname "\$0") würde Fehler wie den zip-Pfadfehler strukturell verhindern.
S5	Jest 29 + ts-jest 29 — koordiniertes Upgrade ausstehend	Niedrig	Niedrig	Jest 30 + ts-jest 30 sind verfügbar, müssen koordiniert gehoben werden. Kein funktionaler Impact.
S6	Keine Positions-Logik in der Heuristik	Niedrig	Mittel	Empfehlungen unterscheiden nicht zwischen Support, Carry, Jungler. Ein ADC erhält die gleichen Empfehlungen wie ein Support.

12. Glossar

Begriff	Definition
---------	------------

ADC	<i>Attack Damage Carry</i> — Champion-Rolle, die primär physischen Schaden austeilt
AP	<i>Ability Power</i> — magische Schadensquelle in League of Legends
AD	<i>Attack Damage</i> — physische Schadensquelle in League of Legends
AsyncAPI	Spezifikationsstandard für ereignisgesteuerte APIs (analog zu OpenAPI für REST)
Bridge	Der Node.js/TypeScript-Prozess, der zwischen der Riot-API und der Flutter-App vermittelt
CC	<i>Crowd Control</i> — Fähigkeiten, die Gegner verlangsamen, betäuben oder immobilisieren
CompProfile	Interne Datenstruktur, die die Zusammensetzung des Gegner-Teams beschreibt (AP-Ratio, AD-Ratio, CC-Score, Heal-Score)
Cooltdown	Wartezeit zwischen zwei aufeinanderfolgenden LLM-Aufrufen (Standard: 7 Minuten)
Data Dragon	Offizielle statische Datenquelle von Riot Games für Item- und Champion-Informationen
EventDetector	Bridge-Komponente, die durch Vergleich zweier Spielzustände Ereignisse erkennt
GameEvent	Typisiertes Ereignis in der Bridge (z. B. <code>GAME_STARTED</code> , <code>ITEM_PURCHASED</code>)
Heuristik	Regelbasiertes System zur Item-Empfehlung ohne externe API-Abhängigkeit
iSAQB	<i>International Software Architecture Qualification Board</i> — Zertifizierungsgremium für Software-Architekten
Live Client API	Lokaler HTTP-Server des League-of-Legends-Clients, der Echtzeit-Spielzustand bereitstellt
LLM	<i>Large Language Model</i> — KI-Modell für natürlichsprachige Texterzeugung (Claude, GPT, Gemini)
LlmProvider	Interface und Adapter-Pattern für alle unterstützten LLM-Anbieter
minifyGameState	Funktion, die den Spielzustand auf eine kompakte Zeichenkette reduziert, um LLM-Token-Kosten zu minimieren
MSIX	Windows-App-Paketformat für den Microsoft Store und Sideloadung
ParsedGameState	Normalisierter Spielzustand nach dem Parser — Grundstruktur für alle Folgekomponenten
pkg / @yao-pkg/pkg	Tool zum Kompilieren von Node.js-Anwendungen in plattformspezifische Binaries
Provider (Flutter)	State-Management-Pattern (ChangeNotifier + Provider-Package)

Retry with Backoff	Strategie zur Fehlerbehandlung: Wiederholungsversuche mit wachsenden Wartezeiten
WsMessage	Gemeinsames Nachrichtenformat zwischen Bridge und Flutter App (JSON over WebSocket)
WsService	Flutter-Dienst (<code>ChangeNotifier</code>), der die WebSocket-Verbindung zur Bridge verwaltet
Zod	TypeScript-Bibliothek zur Schema-Deklaration und Runtime-Validierung